Politecnico di Milano

Ingegneria del Software – a.a. 2009/10

Appello del 23 Luglio 2010

Cognome
Nome
Matricola

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare **solo i fogli distribuiti** utilizzando il **retro** delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. **non** verranno **in nessun caso** presi in considerazione. È possibile scrivere **in matita**.
3. È possibile consultare liberamente libri, manuali o appunti. È **proibito** l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, ecc.).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. Tempo a disposizione: 2h.
6. Punteggio totale a disposizione: 100 punti nominali. La sufficienza si raggiunge indicativamente con 60 punti.

Esercizio 1: .....

Esercizio 2: .....

Esercizio 3: .....

Totale .....

Esercizio 1 (punti 25)

E' data l'astrazione procedurale **calcolo**. Il metodo riceve come parametro un array x di almeno 2 interi e restituisce un array di interi, i cui elementi sono così costituiti:
il primo elemento è la somma di tutti gli elementi di x, il secondo è la somma dei primi x.length -1 elementi di x, il terzo dei primi x.length -2 ecc.

a- Si scrivano la pre- e post-condizione del metodo evidenziando tutti gli elementi significativi

SOL:

```
//@ requires (x != null) && (x.length >= 2)


/*@ ensures \result != null && \result.length == x.length &&
  @      (\forall int i; i >=0 && i < x.length; (\result[i]
  @                == (\sum int j; j >= 0 && j< (x.length - i); x[j])))
  @*/
```

static int[] calcolo(int[] x)

b- Si identifichino le eccezioni necessarie per rendere totale il metodo e si scrivano le opportune modifiche alla specifica di cui al punto (a).

SOL:

```
//@ requires true
/*@ ensures ((x != null) && (x.length >= 2)) =>
  @      \result != null && \result.length == x.length &&
  @                (\forall int i; i >=0 && i < x.length; \result[i]
  @                          == (\sum int j; j >= 0 && j< (x.length - i); x[j]))
  @*/
//@ signals (NullPointerException e) (x == null)
//@ signals (ArrayTooSmallException e) ((x != null) && (x.length < 2))
```

Esercizio 2 (punti 50)

Si consideri la seguente specifica di una classe HeapInt.

```
public class HeapInt{
/* OVERVIEW: Collezione mutabile di oggetti, di tipo intero. */

//@ensures (*inizializza this alla HeapInt vuota*).
public HeapInt ()
//osservatori puri:

//@ensures (* \result è la cardinalità di x in this, ossia il suo numero di comparse *)
public /*@ pure @*/ int occurences(int  x)

//@ensures (* \result è la cardinalità di this, contando ogni elemento con la propria cardinalità *)
public /*@ pure @*/ int size()

//@requires size()>0;
//@ensures (\forall int y; occurrences(y)>0; \result>=y)
public /*@ pure @*/ int max()

//mutators:
//@ensures (* aggiunge una comparsa di x a this *);
public void insert(int x)

//@requires size()>0;
//@ ensures (* elimina e restituisce una comparsa dell'elemento massimo da this  *)
public int extract()
}
```

Domanda A) Specificare in JML le postcondizioni dei metodi insert e extract

SOL:

```
//@ensures occurrences(x) ==\old(occurrences(x)+1) && size() == \old(size()+1) &&

//@(\forall int y; x!=y; occurrences(y) == \old(occurrences(y));

public void insert(int x)

//@ensures \ result == \old(max()) && occurrences(\result) == \old(occurrences(\result)-1) &&

//@ size() == \old(size()-1) && (\forall int y; \result!=y; occurrences(y) == \old(occurrences(y));

public int extract()
```

Si consideri ora una classe HeapInt2, identica a HeapInt ma che modifica il metodo extract in modo che questo elimini tutte le comparse dell'elemento massimale.
Domanda B): Specificare in JML questa versione del metodo extract(),

Sol:

```
//@ensures \ result == \old(max()) && occurrences(\result) == 0 &&

//@ size() == \old(size()-occurrences(\result)) &&

//@(\forall int y; \result!=y; occurrences(y) == \old(occurrences(y));

public int extract()
```

Domanda C) Dimostrare analiticamente, in base alle specifiche date, che HeapInt2 non può essere erede di HeapInt senza violare il principio di sostituzione.

SOL:
La dimostrazione è per assurdo. Detta post1 la postcondizione di HeapInt.extract() e detta post2 la postcondizione di HeapInt2.extract(), condizione necessaria per la verifica del principio è che post2 ==> post1. Ma se vale post1, allora vale *occurrences(\result)==0*, mentre se vale post2 allora vale *occurrences(\result)==\old(occurrences(\result)-1).*
Per uno heap in cui l'elemento max() ha cardinalità 2, da post1 deriva che *occurrences(\result)==0*, mentre da post2 deriva che *occurrences(\result)==1.* Allora se post2 implicasse post1 si dedurrebbe una contraddizione.

Si consideri nuovamente la classe *HeapInt* della Domanda A. È data *un'implementazione* realizzata nel modo seguente. Un attributo *heap* di tipo ArrayList memorizza gli elementi di this come Integer, senza ripetizioni, in ordine decrescente. Un attributo *comparse* di tipo ArrayList memorizza in *comparse.get(i)* il numero di comparse dell'elemento *heap.get(i).* Il numero di comparse non può mai essere nullo o negativo.
Il rep quindi è:

```
private ArrayList<Integer> heap;
private ArrayList<Integer> comparse;
```

Data la seguente implementazione del costruttore:

```
public  HeapInt() {
        heap = new ArrayList<Integer>;
        comparse = new ArrayList<Integer>;
}
```

Domanda D) Scrivere l'invariante di rappresentazione di HeapInt.
SOL:

```
//@private invariant heap !=null && comparse != null && heap.size() == comparse.size() &&
//@(\forall int i; 0< i && i< heap.size(); heap.get(i-1) > heap.get(i)) &&
//@(\forall int i; 0< i && i< comprase.size(); comparse.get(i) >0);
```

NB: la condizione che gli elementi di heap siano tutti distinti deriva dal fatto che sono in ordine strettamente decrescente.

Domanda E) Completare la seguente implementazione del metodo extract()

```
public int extract() {
    int  massimo = this.heap.get(0); // memorizza in massimo il valore da restituire
    ……..comparse.set(0,comparse-get(0)-1);
    ……..if (comparse.get(0)==0 {
    ……..        comparse.remove(0);
    ……..        heap.remove(0);
    …….. }
    return massimo;
}
```

Esercizio 3 (punti 25)

Implementare il seguente metodo iteratore *elementi*() della classe HeapInt dell'Es. 2. Si suggerisce di descrivere la struttura delle eventuali classi aggiuntive rispetto a HeapInt.

//@ensures (restituisce un iteratore agli elementi di this, in ordine decrescente dal più grande al*
//@ più piccolo. Gli elementi che compaiono ripetutamente sono restituiti un numero di volte pari
//@ alla loro cardinalità.);*

public Iterator<Integer> elementi()

SOL:

```
public class HeapInt {
	private ArrayList<Integer> heap;
	private ArrayList<Integer> comparse;
	public Iterator<Integer> elementi() {return new HeapIterator ();}
	private class HeapIterator implements Iterator<Integer> {
		private Integer current;
		private int count;
		private Integer next;
		public HeapIterator () {
			if (heap.size() == 0) {
				this.count = comparse.get(0);
				this.current = heap.get(0);
			}
		}
		public boolean hasNext() {
			if (current != null)
				return true;
		}
		public Integer next() throws NoSuchElementException {
			if (! hasNext()) {
				throw new NoSuchElementException ();
			}
			next = current;
			count --;
			if (!count) {
				int nuovoIndice = heap.indexOf(current) + 1;
				if (nuovoIndice == heap.size())
					current = null;
				else {
					current = heap.get(nuovoIndice);
					count = comparse.get(nuovoIndice);
				}
			}
			return next;
		}
		public void remove() throws IllegalStateException    {
			int index;
			if (next == null)  throw new IllegalStateException();
			index = heap.indexOf(next);
			if (comparse.get(index) == 1) {
				comparse.remove(index);
				heap.remove(index);
			}
			next = null;
		}
	}
}
```